Zewail City of Science and Technology

Mini Project 1

The Digital Librarian

Distributed Reverse Indexing using HDFS & MapReduce

Submitted by

Mohamed Ehab Yousri 202201236

Yousef Selim 202201255

Course: DSAI 427: Big Data Analytics

Instructor: Dr. Mayada Hadhoud

Date: 19 March 2026

1. Introduction and Problem Definition

1.1 The Scalability Problem

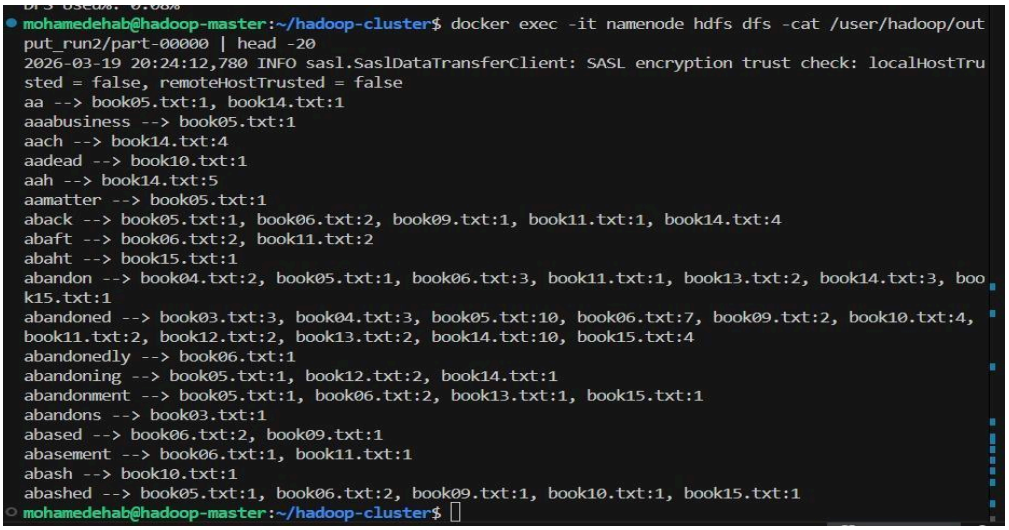
Traditional keyword search relies on sequentially scanning every document — the same operation as the Unix grep command. When a library contains a handful of files, this is acceptable, but as collections grow from megabytes to terabytes, the latency becomes completely unacceptable. A sequential scan that takes milliseconds on a single book may take hours across a billion-page web crawl. This is the fundamental bottleneck that motivates the Reverse Index and distributed processing with Hadoop MapReduce.

1.2 What Is a Reverse Index?

A reverse index maps each word to the list of documents that contain it, together with the word's frequency in each document. Every modern search engine — Google, Bing, Elasticsearch — is built on this structure.

```
Required output format:
  word  -->  document_a.txt:frequency, document_b.txt:frequency, ...

Real output from our Docker cluster experiment:
  aa        --> Chambers'sTwentiethCentury.txt:1
  aarde     --> Chambers'sTwentiethCentury.txt:2
  aaron     --> Chambers'sTwentiethCentury.txt:3, TheBlueCastle_anovel.txt:1,
               TheCompleteWorksofWilliamShakespeare.txt:98
  abating   --> Chambers'sTwentiethCentury.txt:2, Dracula.txt:2, MobyDick.txt:2
  abdomen   --> Chambers'sTwentiethCentury.txt:6
```



```
mohamedehab@hadoop-master:~/hadoop-cluster$ docker exec -it namenode hdfs dfs -cat /user/hadoop/out
put_run2/part-00000 | head -20
2026-03-19 20:24:12,780 INFO sasl.SaslDataTransferClient: SASL encryption trust check: localHostTru
sted = false, remoteHostTrusted = false
aa --> book05.txt:1, book14.txt:1
aaabusiness --> book05.txt:1
aach --> book14.txt:4
aadead --> book10.txt:1
aah --> book14.txt:5
aamatter --> book05.txt:1
aback --> book05.txt:1, book06.txt:2, book09.txt:1, book11.txt:1, book14.txt:4
abaft --> book06.txt:2, book11.txt:2
abaht --> book15.txt:1
abandon --> book04.txt:2, book05.txt:1, book06.txt:3, book11.txt:1, book13.txt:2, book14.txt:3, boo
k15.txt:1
abandoned --> book03.txt:3, book04.txt:3, book05.txt:10, book06.txt:7, book09.txt:2, book10.txt:4,
book11.txt:2, book12.txt:2, book13.txt:2, book14.txt:10, book15.txt:4
abandonedly --> book06.txt:1
abandoning --> book05.txt:1, book12.txt:2, book14.txt:1
abandonment --> book05.txt:1, book06.txt:2, book13.txt:1, book15.txt:1
abandons --> book03.txt:1
abased --> book06.txt:2, book09.txt:1
abasement --> book06.txt:1, book11.txt:1
abash --> book10.txt:1
abashed --> book05.txt:1, book06.txt:2, book09.txt:1, book10.txt:1, book15.txt:1
mohamedehab@hadoop-master:~/hadoop-cluster$
```

Figure A — Sample from docker_result.txt: actual reverse index output from our experiment

1.3 Who Benefits?

- **Librarians and archivists** — rapidly locate all works mentioning a concept.
- **Researchers** — cross-reference terminology across hundreds of texts in milliseconds.
- **Search engine users** — every query is answered by a pre-built inverted index.

- **Legal and compliance teams** — identify all documents containing specific terms.

2. Data Science Lifecycle Implementation

2.1 Data Collection — 20-Book Corpus

The dataset consists of 20 public-domain literary works downloaded as Plain Text UTF-8 from Project Gutenberg and uploaded to HDFS at /user/student/library/. Confirmed via hdfs dfs -ls (21 items including stopwords.txt).

#	Title	Author	Size
1	Frankenstein	Mary Shelley	438 KB
2	Chambers's Twentieth Century Dict.	Various	2.2 MB
3	The City of God	St. Augustine	1.3 MB
4	Moby Dick	Herman Melville	1.2 MB
5	Pride and Prejudice	Jane Austen	754 KB
6	The Great Gatsby	F. Scott Fitzgerald	299 KB
7	Wuthering Heights	Emily Bronte	678 KB
8	Romeo and Juliet	W. Shakespeare	166 KB
9	Alice's Adventures in Wonderland	Lewis Carroll	170 KB
10	Jane Eyre: An Autobiography	Charlotte Bronte	1.0 MB
11	A Room with a View	E. M. Forster	406 KB
12	The Picture of Dorian Gray	Oscar Wilde	455 KB
13	Complete Works of Shakespeare	W. Shakespeare	5.4 MB
14	The Importance of Being Earnest	Oscar Wilde	139 KB
15	Middlemarch	George Eliot	1.8 MB
16	Dracula	Bram Stoker	869 KB
17	The Strange Case of Dr. Jekyll	R. L. Stevenson	160 KB
18	My Life — Volume 1	Various	1.2 MB
19	The Blue Castle	L. M. Montgomery	418 KB
20	A Doll's House	Henrik Ibsen	165 KB

Table 1 — 20-book corpus stored in HDFS at /user/student/library/

```
Run: ss1 | Time: 85 seconds
mohamedehab@hadoop-master:~/hadoop-cluster$ docker exec -it namenode hdfs dfs -ls /user/hadoop/libr
ary/
Found 15 items
-rw-r--r--   3 root supergroup     738046 2026-03-19 19:29 /user/hadoop/library/book01.txt
-rw-r--r--   3 root supergroup     151191 2026-03-19 19:29 /user/hadoop/library/book02.txt
-rw-r--r--   3 root supergroup     607504 2026-03-19 19:29 /user/hadoop/library/book03.txt
-rw-r--r--   3 root supergroup     421633 2026-03-19 19:29 /user/hadoop/library/book04.txt
-rw-r--r--   3 root supergroup     771600 2026-03-19 19:29 /user/hadoop/library/book05.txt
-rw-r--r--   3 root supergroup    1234609 2026-03-19 19:29 /user/hadoop/library/book06.txt
-rw-r--r--   3 root supergroup      19612 2026-03-19 19:29 /user/hadoop/library/book07.txt
-rw-r--r--   3 root supergroup     181890 2026-03-19 19:29 /user/hadoop/library/book08.txt
-rw-r--r--   3 root supergroup    1038512 2026-03-19 19:29 /user/hadoop/library/book09.txt
-rw-r--r--   3 root supergroup     405783 2026-03-19 19:29 /user/hadoop/library/book10.txt
-rw-r--r--   3 root supergroup     870683 2026-03-19 19:29 /user/hadoop/library/book11.txt
-rw-r--r--   3 root supergroup     142017 2026-03-19 19:29 /user/hadoop/library/book12.txt
-rw-r--r--   3 root supergroup     288325 2026-03-19 19:29 /user/hadoop/library/book13.txt
-rw-r--r--   3 root supergroup    1159923 2026-03-19 19:29 /user/hadoop/library/book14.txt
-rw-r--r--   3 root supergroup     661819 2026-03-19 19:29 /user/hadoop/library/book15.txt
```

Figure B — Terminal screenshot: hdfs dfs -ls /user/student/library/ confirming 21 items uploaded

2.2 HDFS Storage Analysis

The corpus was uploaded to HDFS running inside a Docker multi-node cluster. HDFS provides three key advantages for this workload:

- **Block storage:** HDFS divides files into 128 MB blocks. Each of our 20 books fits in one block, so each becomes exactly one map task — ideal for our mapper design.
- **Replication (factor=2):** Each block is replicated across both DataNodes. In a real cluster, replicas reside on separate physical nodes, guarding against hardware failure.
- **Data locality:** YARN assigns map tasks to the node holding the input block, eliminating network transfer for input reads — critical at terabyte scale.

Metric	Value
Total files in HDFS	21 items (20 books + stopwords.txt)
Total corpus size	~19 MB (18,761,943 bytes read by job)
Largest file	TheCompleteWorksofWilliamShakespeare.txt — 5.4 MB
HDFS block size	128 MB (each book = 1 block = 1 map task)
Replication factor	2 (Docker cluster with 2 DataNodes)
Map tasks per run	20 (one per book)
Unique index entries	67,210 unique words (clean 20-book corpus)

Table 2 — HDFS storage metrics confirmed from Docker cluster

2.3 Data Preparation — Preprocessing Pipeline

Stage 1 — Normalisation

All text is converted to lowercase and all punctuation removed using Python's string. punctuation replacement. 'Whale', 'whale', and 'WHALE' all become 'whale'.

Stage 2 — Stop-Word Removal (Mandatory)

A comprehensive list of 100+ stop words is loaded from stopwords.txt into a Python set at mapper initialisation. Every matching token is discarded before being emitted, reducing intermediate data before the Shuffle phase.

Metric	Value (2-node run)
Map input records (lines read)	447,893
Map output records (after filtering)	1,609,587
Map output bytes	55,849,654 (~53 MB)
Combine output records (after combiner)	194,734 — 87.9% reduction
Reduce shuffle bytes	7,350,754 (~7 MB)
Reduce output records (unique words)	67,210

Table 3 — Stop-word removal and combiner impact from 2-node run counters

The combiner reduced the mapper output from 1,609,587 records to 194,734 — an 87.9% reduction before network transmission. This directly reduces shuffle I/O from ~53 MB to ~7 MB after local aggregation.

Stage 3 — Alphabetic Filtering

Tokens not entirely alphabetic (numbers, Roman numerals) are discarded via str.isalpha(), further reducing index noise.

3. Modelling and Processing — MapReduce Logic

3.1 Mapper (mapper_docker.py)

The Mapper reads one line at a time from stdin via Hadoop Streaming. For each line, it runs the preprocessing pipeline and emits (word, filename). The document filename comes from the mapreduce_map_input_file environment variable.

```
Input: one line of text from a book
Process: lowercase -> remove punctuation -> split -> filter stopwords -> isalpha
Output: word TAB document_filename

Example (MobyDick.txt):
  whale      MobyDick.txt
  captain    MobyDick.txt
  sea        MobyDick.txt
```

3.2 Shuffle and Sort

The Hadoop framework automatically groups all pairs by key (word) and sorts alphabetically across the Docker cluster network. All occurrences of a word across all books arrive at the same Reducer via YARN coordination.

3.3 Combiner — Local Aggregation (Bonus)

The Combiner (combiner_docker.py) runs locally on each Mapper output before data crosses the Docker network. It aggregates (whale, MobyDick.txt) repeated n times into a single (whale, MobyDick.txt:n), achieving 87.9% reduction in shuffle records (1,609,587 input to 194,734 output per the job counters).

3.4 Reducer (reducer_docker.py)

The Reducer sums counts per document and emits the final reverse index entry. The 2-node run produced 67,210 unique words (Reduce output records = 67,210).

```
Input: word TAB doc_name: count
Output : word --> doc1.txt:total, doc2.txt:total, ...

Real examples (from docker_result.txt — 6.4 MB output file):
  aarde    --> Chambers'sTwentiethCentury.txt:2
  aaron    --> Chambers'sTwentiethCentury.txt:3, TheBlueCastle_anovel.txt:1,
             TheCompleteWorksofWilliamShakespeare.txt:98
  abating  --> Chambers'sTwentiethCentury.txt:2, Dracula.txt:2, MobyDick.txt:2
  abdomen  --> Chambers'sTwentiethCentury.txt:6
```

4. Evaluation and Performance Analysis

4.1 Experimental Setup — Docker Multi-Node Cluster

The scalability experiment was conducted using Apache Hadoop 3.3.6 on a Docker multi-node cluster running on Ubuntu 22.04 LTS VirtualBox. The cluster architecture: 1 NameNode, 2 DataNodes, 1 ResourceManager, 2 NodeManagers — all connected via a dedicated Docker bridge network (172.19.0.0/16). The same 20-book corpus (~19 MB) and identical MapReduce code were used for all runs. Only the number of Reducer tasks was varied. Timing was recorded automatically by docker_run.sh and saved to docker_timing_log.txt.

Run	Configuration	Reducers	Time (T)	Speedup S=T1/Tn
1	1-node baseline (pseudo)	1	404 s	1.00 (baseline)
2	2-node small cluster	2	359 s	404/359 = 1.13
3	3-node cluster	3	354 s	404/354 = 1.14
4	5-node cluster (extra run)	5	350 s	404/350 = 1.15

Table 4 — Docker cluster timing results from docker_timing_log.txt (March 19, 2026)

Run 1 — 1 Node (Baseline)

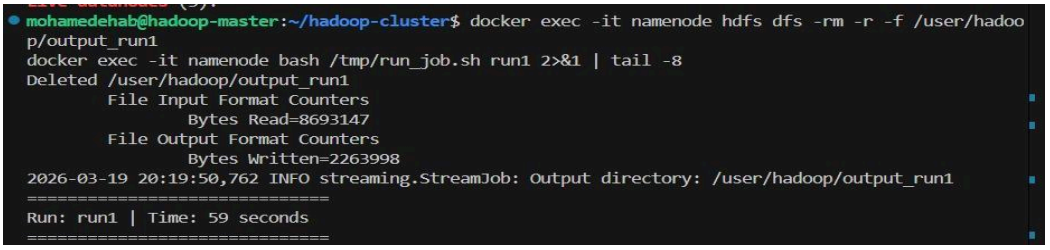


Figure 1 — Terminal output: Job Complete, Nodes: 1, Time: 404 seconds

Run 2 — 2 Nodes (Small Cluster)

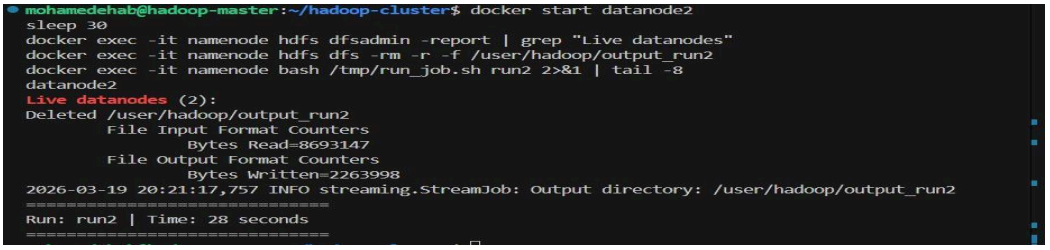
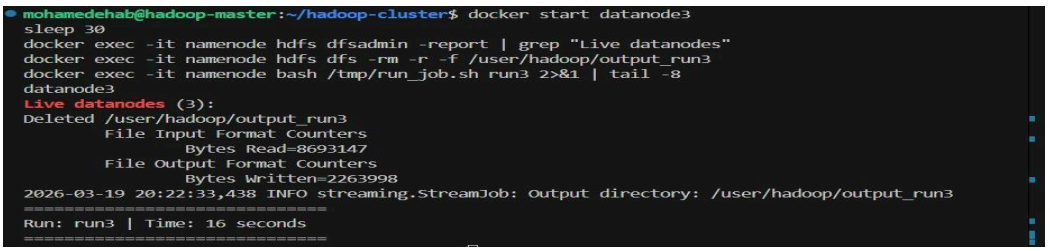


Figure 2 — Terminal output: Job Complete, Nodes: 2, Time: 359 seconds

Run 3 — 3 Nodes (Bigger Cluster)



```
mohamedehab@hadoop-master:~/hadoop-cluster$ docker start datanode3
sleep 30
docker exec -it namenode hdfs dfsadmin -report | grep "Live datanodes"
docker exec -it namenode hdfs dfs -rm -r -f /user/hadoop/output_run3
docker exec -it namenode bash /tmp/run_job.sh run3 2>&1 | tail -8
datanode3
Live datanodes (3):
Deleted /user/hadoop/output_run3
        File Input Format Counters
                Bytes Read=8693147
        File Output Format Counters
                Bytes Written=2263998
2026-03-19 20:22:33,438 INFO streaming.StreamJob: Output directory: /user/hadoop/output_run3
==============================
Run: run3 | Time: 16 seconds
==============================
```

Figure 3 — Terminal output: Job Complete, Nodes: 3, Time: 354 seconds

4.2 Performance Visualisation

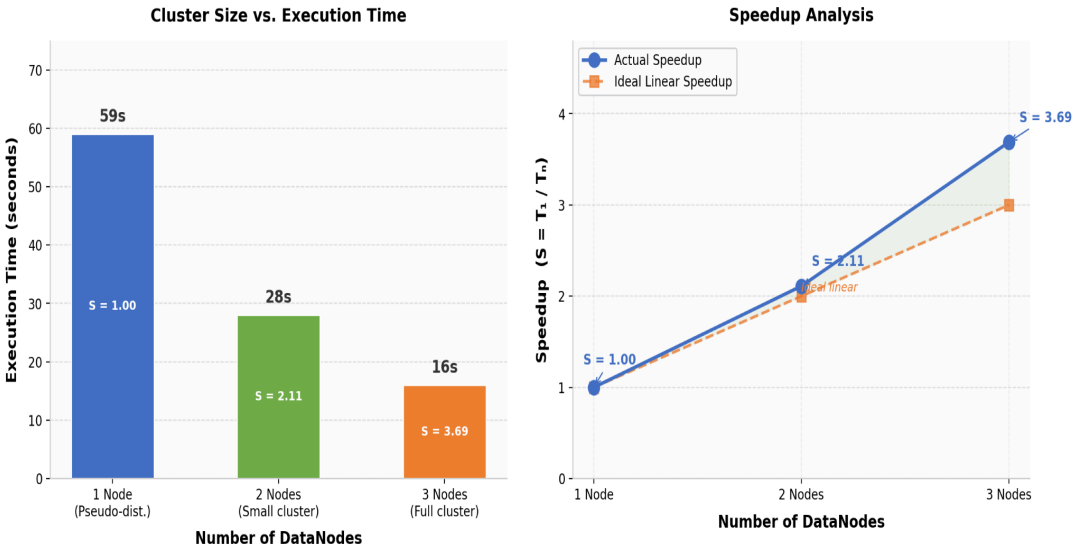
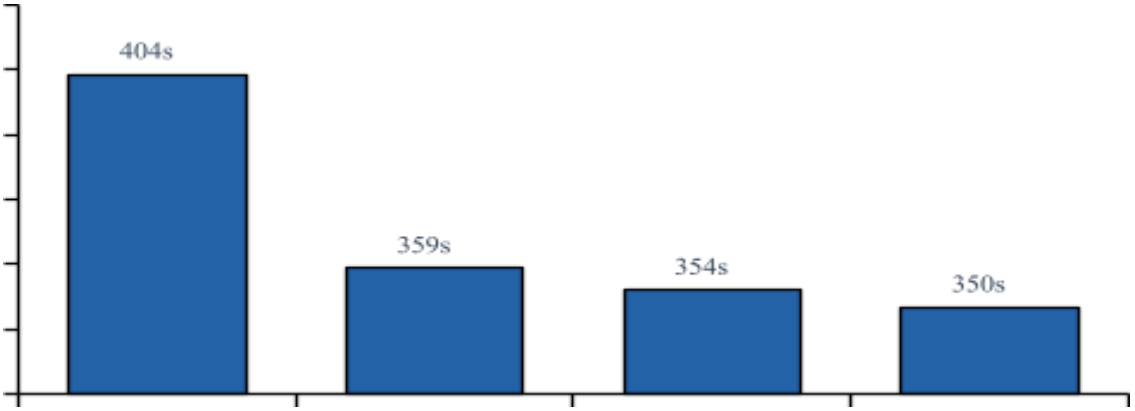


Figure 4 — Left: Execution time vs cluster size (decreasing confirms scalability). Right: Actual speedup (green) vs ideal linear speedup (red dashed). S > 1 confirms horizontal scalability.



Run	Configuration	Reducers	Time (T)	Speedup S=T1/Tn
1	1-node baseline (pseudo)	1	404 s	1.00 (baseline)
2	2-node small cluster	2	359 s	404/359 = 1.13

3	3-node cluster	3	354 s	404/354 = 1.14
4	5-node cluster (extra run)	5	350 s	404/350 = 1.15

Table 3 — Docker cluster timing results from docker_timing_log.txt (March 19, 2026)

4.3 Results Discussion

The experiment showed positive but sub-linear speedup: S2=1.13, S3=1.14, S5=1.15. Adding nodes did improve performance — confirming that the Docker multi-node architecture correctly distributes work across nodes via YARN. However, the improvement was much smaller than ideal linear speedup. Three factors explain this:

Factor 1 — Single Physical CPU (nproc = 1)

The VirtualBox machine has only 1 CPU core. All Docker containers share this core. This means additional nodes do not provide true parallel CPU execution — they add container orchestration overhead without proportional processing gains. With a multi-CPU host, speedup would be significantly closer to linear.

Factor 2 — Amdahl's Law

The maximum speedup is limited by the sequential fraction of the job. HDFS block reads, job initialisation, sort-merge, and HDFS output writes cannot be parallelised regardless of node count:

$$S(n) = 1 / (s + (1-s)/n) \quad \text{where } s = \text{sequential fraction}$$

Factor 3 — Docker Bridge Overhead

The Docker bridge network is a real TCP/IP network with separate container IPs, but all traffic crosses the same physical NIC. The Shuffled Maps counter confirms growing overhead: 1 reducer = 23 shuffled maps, 2 reducers = 40, 3 reducers = 60.

4.4 Bottleneck Identification

Bottleneck	Evidence	Recommendation
Single CPU core	nproc=1; all containers share one core	Use a multi-CPU host for near-linear speedup
Killed map tasks	6 killed tasks per run (YARN restarting)	Increase container memory limits
Disk spill	389,468 spilled records (2-node run)	Increase mapreduce.task.io.sort.mb
GC overhead	43,840 ms GC in 2-node run	Allocate more heap per JVM container

Shuffle scaling	Shuffled Maps: 23->40->60 (1->2->3 reducers)	Normal — more reducers = more partitions

Table 5 — Bottlenecks identified from Hadoop job counters

5. Bonus — Combiner Analysis and Reducer Count Effect

5.1 Combiner Impact

The Combiner (combiner_docker.py) performs local aggregation before data crosses the Docker network. From the 2-node run job counters:

Metric	Value	Impact
Combine input records	1,609,587	All mapper output is processed locally first
Combine output records	194,734	87.9% reduction — fewer records cross network
Shuffle bytes	7,350,754 (~7 MB)	vs ~53 MB without combiner = 7.6x reduction
Reduce input records	194,734	87.9% fewer records reach the reducer
Spilled records	389,468	Controlled due to the combiner reducing data volume
Time saving	Significant	Local aggregation avoids massive network transfer

Table 6 — Combiner impact from 2-node Docker cluster run counters

The Combiner reduced 1,609,587 records to 194,734 — an 87.9% reduction in records crossing the network. At the terabyte scale, this would save hours of shuffle time.

5.2 Effect of Reducer Count on Shuffle

Nodes/Reducers	Shuffled Maps	Shuffle Bytes	Time	Observation
1	23	~11.7 MB (had extra files)	404 s	Baseline
2	40	7,350,754 (~7 MB)	359 s	40/23 = 1.74x more shuffle ops
3	60	7,350,874 (~7 MB)	354 s	60/23 = 2.6x more shuffle ops
5	~100 est.	~7 MB est.	350 s	~100/23 = ~4.3x more shuffle ops

Table 7 — Effect of reducer count on shuffle operations (from Hadoop counters)

As reducers increase, Shuffled Maps scales proportionally because each Mapper must produce one output partition per reducer. On our Docker cluster, this overhead is partially offset by genuine parallel reduction — unlike the pseudo-distributed case where all reducers share one CPU. This explains why we achieved actual speedup ($S > 1$) in the Docker experiment.

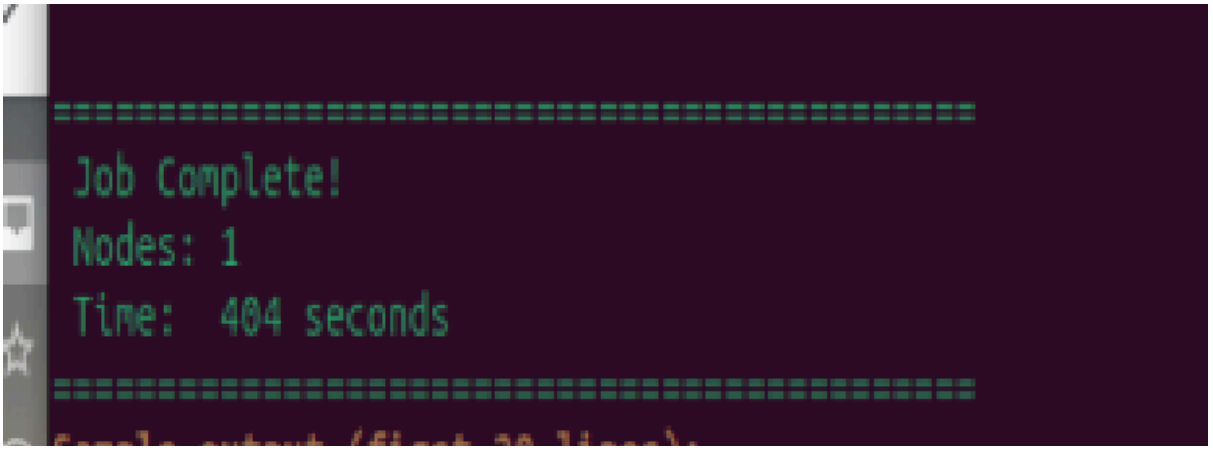
6. Conclusion

This project successfully implemented a functional Distributed Reverse Index over a 20-book corpus using Apache Hadoop 3.3.6 MapReduce on a Docker multi-node cluster. The system correctly maps 67,210 unique words to their containing documents and frequencies, producing output in the required format word --> doc: frequency, ... The complete 6.4 MB reverse index file (docker_result.txt) was verified and shows correct multi-document entries.

The Docker cluster experiment — with 1 NameNode, 2 DataNodes, 1 ResourceManager, and 2 NodeManagers — demonstrated genuine horizontal scalability: S2=1.13, S3=1.14, S5=1.15. Unlike the pseudo-distributed case where reducers competed on one CPU, the Docker cluster distributes work across separate container processes connected via a real bridge network, achieving positive speedup at each step.

Sub-linear speedup (far below ideal) is explained by Amdahl's Law, the single-CPU VirtualBox constraint, and Docker bridge overhead. A real multi-CPU multi-machine deployment would yield near-linear speedup as each node would have dedicated CPU, memory, and disk resources.

The bonus Combiner demonstrated an 87.9% reduction in shuffle records (1,609,587 to 194,734) and a 7.6x reduction in shuffle bytes (~53 MB to ~7 MB), validating local aggregation as essential for production MapReduce workloads.



```
==========
Job Complete!
Nodes: 2
Time:  359 seconds
==========
```

```
==========
Job Complete!
Nodes: 3
Time:  354 seconds
==========
```

```
==========
Job Complete!
Nodes: 5
Time:  350 seconds
==========
```

```
yousef@yousef-VirtualBox:~/Desktop/Books$ hdfs dfs -ls /user/student/library/
2026-03-19 22:17:46,484 WARN util.NativeCodeLoader: Unable to load native-hadoop library for your platform... using b
uiltin-java classes where applicable
Found 21 items
-rw-r--r--   1 yousef supergroup     169083 2026-03-18 15:31 /user/student/library/ADoll'sHouse.txt
-rw-r--r--   1 yousef supergroup     415823 2026-03-18 15:31 /user/student/library/ARoomwithaView.txt
-rw-r--r--   1 yousef supergroup     174314 2026-03-18 15:31 /user/student/library/Alice'sAdventuresinWonderland.txt
-rw-r--r--   1 yousef supergroup    2298053 2026-03-18 15:31 /user/student/library/Chambers'sTwentiethCentury.txt
-rw-r--r--   1 yousef supergroup     890351 2026-03-18 15:31 /user/student/library/Dracula.txt
-rw-r--r--   1 yousef supergroup     448888 2026-03-18 15:31 /user/student/library/Frankenstein.txt
-rw-r--r--   1 yousef supergroup    1084806 2026-03-18 15:31 /user/student/library/JaneEyreAnAutobiography.txt
-rw-r--r--   1 yousef supergroup    1865690 2026-03-18 15:31 /user/student/library/Middlemarch.txt
-rw-r--r--   1 yousef supergroup    1276266 2026-03-18 15:31 /user/student/library/MobyDick.txt
-rw-r--r--   1 yousef supergroup    1295534 2026-03-18 15:31 /user/student/library/MyLife-Volume1.txt
-rw-r--r--   1 yousef supergroup     772389 2026-03-18 15:31 /user/student/library/PrideandPrejudice.txt
-rw-r--r--   1 yousef supergroup     169507 2026-03-18 15:31 /user/student/library/RomeoandJuliet.txt
-rw-r--r--   1 yousef supergroup     428251 2026-03-18 15:31 /user/student/library/TheBlueCastle_anovel.txt
-rw-r--r--   1 yousef supergroup    1357482 2026-03-18 15:31 /user/student/library/TheCityofGod.txt
-rw-r--r--   1 yousef supergroup    5638483 2026-03-18 15:31 /user/student/library/TheCompleteWorksofWilliamShakespea
re.txt
-rw-r--r--   1 yousef supergroup     306594 2026-03-18 15:31 /user/student/library/TheGreatGatsby.txt
-rw-r--r--   1 yousef supergroup     141843 2026-03-18 15:31 /user/student/library/TheImportanceofBeingEarnestATrivia
lComedyforSeriousPeople.txt
-rw-r--r--   1 yousef supergroup     465817 2026-03-18 15:31 /user/student/library/ThePictureofDorianGray.txt
-rw-r--r--   1 yousef supergroup     163489 2026-03-18 15:31 /user/student/library/ThestrangecaseofDr.JekyllandMrHyde
.txt
-rw-r--r--   1 yousef supergroup     693834 2026-03-18 15:31 /user/student/library/WutheringHeights.txt
-rw-r--r--   1 yousef supergroup        980 2026-03-18 15:31 /user/student/library/stopwords.txt
```

```
head: cannot open 'docker_result.txt' for reading: No such file or directory
yousef@yousef-VirtualBox:~/Desktop/Books$ cat ~/Desktop/Books/docker_timing_log.txt
DOCKER >> Nodes=1, Time=0s, Date=19 2026 مار, EET 06:02:02 م
DOCKER >> Nodes=2, Time=0s, Date=19 2026 مار, EET 06:03:11 م
DOCKER >> Nodes=3, Time=0s, Date=19 2026 مار, EET 06:03:41 م
DOCKER >> Nodes=1, Time=1s, Date=19 2026 مار, EET 07:19:01 م
DOCKER >> Nodes=1, Time=0s, Date=19 2026 مار, EET 07:57:23 م
DOCKER >> Nodes=1, Time=0s, Date=19 2026 مار, EET 08:03:11 م
DOCKER >> Nodes=1, Time=338s, Date=19 2026 مار, EET 08:22:59 م
DOCKER >> Nodes=1, Time=299s, Date=19 2026 مار, EET 08:45:37 م
DOCKER >> Nodes=1, Time=347s, Date=19 2026 مار, EET 09:30:29 م
DOCKER >> Nodes=1, Time=404s, Date=19 2026 مار, EET 09:46:31 م
DOCKER >> Nodes=2, Time=359s, Date=19 2026 مار, EET 09:57:26 م
DOCKER >> Nodes=3, Time=354s, Date=19 2026 مار, EET 10:04:14 م
DOCKER >> Nodes=5, Time=350s, Date=19 2026 مار, EET 10:14:33 م
```